

Ch9 函数

对应《C Primer Plus（第6版）》第9章 · 本章目标：40 分钟读懂 + 60 分钟练完 · 前置：Ch1–Ch7 · 学完本章，Ch10 的数组与指针才能继续

9.1 先看一个真实场景：为什么需要函数

你写了一个「把 10 个 LED 依次点亮」的程序，现在要再点亮 20 个、30 个——把点亮逻辑复制三遍？代码会越堆越长，改一个细节要改三处。函数就是**给一段代码起名字、随叫随到**的工具：

- **复用**：同一段逻辑写一次，需要时调用多次；
- **分工**：大程序拆成小函数，各管一段，出错好定位；
- **可读**：main 里全是「做什么」（turn_on_led(3)），而不是「怎么做」。

电子信息专业的视角：驱动库、算法库（FFT、滤波）全部是「函数集合」。你之后读学长代码、用厂商 SDK，第一步就是找它的函数原型。

9.2 函数的定义：返回值类型 + 名字 + 参数 + 函数体

函数定义长这样（和 main 同级的兄弟函数）：

```
/* ch9_01_add.c — 最简单的函数：两数相加 */
#include <stdio.h>

int add(int a, int b)      /* 返回类型 int; 参数 a、b 都是 int */
{
    return a + b;          /* return 把结果交回给调用者 */
}

int main(void)
{
    int r = add(3, 5);      /* 调用：实参 3 和 5 传给形参 a 和 b */
    printf("%d\n", r);     /* 输出 8 */
    return 0;
}
```

编译：gcc -Wall -Wextra -o add.exe ch9_01_add.c 运行：.\add.exe → 8

部件	说明	示例
----	----	----

返回类型	函数交回的值是什么类型；不需要交回就用 <code>void</code>	<code>int / void</code>
函数名	动词化命名，让别人一看就知道干什么	<code>add / turn_on_led</code>
参数列表	函数需要的「输入」；没有就用 <code>void</code>	<code>(int a, int b) / (void)</code>
函数体	<code>{ }</code> 里的语句，最后通常 <code>return</code>	<code>{ return a + b; }</code>

新手坑 1：形参和实参是两个东西。定义里写的是**形参**（`a`、`b` 是占位符），调用时传的是**实参**（`3`、`5` 是真值）。调用那一刻，实参的值被**复制**给形参——函数内部改形参，外面的实参不受影响（9.4 细讲）。

9.3 函数原型：先声明，后使用

编译器读代码是**从上到下**的。如果 `main` 在 `add` 前面，编译器看到 `add(3, 5)` 时还不知道 `add` 是什么 → 报「隐式声明」错误（C99 之后直接拒绝编译）。解决办法：在调用之前放一行**函数原型**：

```
/* ch9_02_proto.c - 函数原型先行 */
#include <stdio.h>

int add(int a, int b);    /* 函数原型：只声明签名，不写函数体 */

int main(void)
{
    printf("%d\n", add(3, 5));
    return 0;
}

int add(int a, int b)      /* 函数定义可以放后面 */
{
    return a + b;
}
```

习惯：所有函数原型集中在 `main` 前面（或放头文件），函数定义随意排。这也是你读别人代码时的第一站。

9.4 参数传递：值传递（`swap` 为什么不行）

参数是**值传递**——传进去的是副本。经典的「交换两个数」陷阱：

```

/* ch9_03_swap.c - 为什么 swap 没生效? */
#include <stdio.h>

void swap(int a, int b)    /* 形参 a、b 是实参的副本 */
{
    int t = a;
    a = b;
    b = t;                /* 只交换了副本 */
}

int main(void)
{
    int x = 1, y = 2;
    swap(x, y);
    printf("x=%d y=%d\n", x, y);    /* 输出 x=1 y=2, 没换! */
    return 0;
}

```

实测输出: x=1 y=2—swap 里的 a、b 是 x、y 的复制品, 交换副本不影响原件。

新手坑 2: 想改外面的变量, 要么传地址 (指针, Ch10 学), 要么让函数 return 新值再赋值。目前阶段: 需要「带出结果」就 return, 如 `x = add(x, 5);`。

9.5 返回值与 void: 函数能交回什么

return 表达式; 把结果交回调用处; void 函数不交回值 (但可以 return; 提前结束):

```

/* ch9_04_return.c - 返回值与提前返回 */
#include <stdio.h>

int abs_val(int n)        /* 绝对值 */
{
    if (n < 0) return -n; /* 提前返回 */
    return n;
}

void report(int n)        /* void: 只做事, 不交回 */
{
    printf("abs(%d) = %d\n", n, abs_val(n));
}

int main(void)
{
    report(-7);           /* 输出 abs(-7) = 7 */
    report(4);            /* 输出 abs(4) = 4 */
}

```

```
    return 0;
}
```

返回类型要对上：return 的类型和函数声明的返回类型不一致，编译器会警告（-Wall 会提醒）。double 函数里 return int 值 → 自动转 double，但反过来 int 函数里 return double 会**截断**（丢小数）。

9.6 函数与数组：传数组 = 传首地址（const 保护）

把整个数组复制给函数太浪费，C 的做法是**只传数组首地址 + 长度**（指针细节 Ch10 展开，这里先会用）：

```
/* ch9_05_array.c - 数组求和函数（Ch10 会用到同款写法）*/
#include <stdio.h>

int sum_array(const int arr[], int n)    /* const: 承诺不修改数组内容 */
{
    int s = 0;
    for (int i = 0; i < n; i++)
        s += arr[i];
    return s;
}

int main(void)
{
    int a[] = {2, 4, 6, 8};
    printf("%d\n", sum_array(a, 4));    /* 输出 20 */
    return 0;
}
```

实测输出：20。arr 形参里写 const int arr[] 是告诉编译器「我只读不改」—误改 arr[i] 会直接报错，防止手滑。

新手坑 3：函数里 sizeof(arr) 不是数组大小！形参 int arr[] 本质是指针，sizeof(arr) 是指针大小（8 字节，64 位机），不是数组字节数。所以必须**同时传长度 n**。这是 C 语言的经典陷阱，面试也爱考。

9.7 递归：函数调用自己（先看懂再写）

递归 = 函数体里调用它自己，配合「终止条件」结束。经典阶乘：

```
/* ch9_06_fact.c - 递归求阶乘 */
#include <stdio.h>

long fact(int n)
{
    if (n <= 1) return 1;    /* 终止条件：不再往下调 */
    return n * fact(n - 1); /* 自己调用自己，规模缩小 */
}
```

```

}

int main(void)
{
    printf("%ld\n", fact(5));    /* 输出 120 */
    return 0;
}

```

实测输出：120（ $5 \times 4 \times 3 \times 2 \times 1$ ）。递归执行路径： $\text{fact}(5) \rightarrow 5 \times \text{fact}(4) \rightarrow 5 \times 4 \times \text{fact}(3) \rightarrow \dots \rightarrow 5 \times 4 \times 3 \times 2 \times 1$ 。

递归三要素：①终止条件（不满足就无限调 → 栈溢出崩溃）；②每次调用规模变小；③能把「缩小后的结果」拼回大结果。写不出递归就用循环，二者等价，别硬撑。

9.8 新手坑集中营（本章最容易错的 4 件事）

- **1. 漏函数原型：**main 在前、函数定义在后，中间没有原型 → 编译报「隐式声明」错误。报错先看是不是函数顺序问题。
- **2. 函数里 sizeof(数组形参)：**拿到的是指针大小不是数组大小——数组参数必须另传长度。
- **3. 忘记 return / 返回类型不匹配：**int 函数漏 return 是**未定义行为**（可能返回垃圾值）；-Wall 会警告「control reaches end of non-void function」——看到立刻补。
- **4. 想用函数改外部变量却用值传递：**swap(x,y) 换不了——要么 return 新值，要么等 Ch10 学指针再传地址。

9.9 一分钟自测（先答后揭，答案见练习册）

- Q1：函数原型和函数定义的区别是什么？
- Q2：void 函数可以写 return; 吗？
- Q3：int sum(const int a[], int n) 里的 const 有什么用？
- Q4：为什么 swap(int a, int b) 换不了外面的两个变量？
- Q5：递归必须有的两个条件是什么？

9.10 练习指引

打开「练习册 · Ch9」：先做 **9-1 填空**（热身）→ **9-2 改错**（先自己找错再对答案，最有价值）→ **9-3/9-4 编程**（自己写，参考代码在参考答案）→ **9-5 专业场景**（写一个 LED 灯效函数，电子信息味）。写完对照参考答案，重点看「常见错法」。